CS361 Coding and Documentation Standards

When you create a program for submission, you must follow these coding and documentation standards.

At the top of your program code, you must provide an extended comment that includes:

- Your name, the course, and the assignment number.
- A brief description of what the program does, including:
  - what is input
  - what processing is performed
  - what is output

Within your program code, you must provide the following:

- Each constant, variable, and function should have a descriptive, meaningful **name**.
  - Constant names should be in ALL CAPS. If the name contains multiple words, use an underscore to separate each word.
  - Variable names should begin with a **lowercase** letter, and be composed of mainly lowercase letters. If the name contains multiple words, you may choose to either use an underscore to separate the words, or to capitalize the beginning of each successive word. But you should choose ***one*** of these two methods, and use it consistently throughout your program.
  - Any ***global*** **constants** should be declared above the main function, at the top of the code.
  - All ***local*** constants and variables should be defined at the **top** of the function, before any other executable lines of code. If a function contains both constant and variable declarations, then all the constants should be declared first, followed by all variable declarations, with a blank line separating the constant declarations from the variable declarations
  - Each constant and variable should be declared on its own code line. If the name is not ***completely*** obvious, a comment must be included, tabbed out to the right of the declaration, describing how the constant/variable will be used.

    For example:

```
const int MAX_INTEREST = 30; // maximum interest rate

int distance,                // distance from object in feet
    time;                    // elapsed time in seconds
```

  - Function names should describe what a function does, using verbs or phrases containing verbs. Function names should follow the same conventions as variable names, except that you may choose to begin them with an **uppercase** letter, if you like, to distinguish them from variable names. If the function name contains multiple

©2013. Regis University

words, you may choose to either use an underscore to separate the words, or to capitalize the beginning of each successive word. But one method must be chosen and used consistently throughout the program.

- A **space** is required between keywords in a program. Extra spaces between words and symbols are ignored by the compiler and should be inserted to improve readability.
  - You should leave a space **before and after** operators, **between** names, **after** commas, and to align code and multiple line statements.

    For example, this code:

```
if (num < 5)
   value = num + (value - 1) / 2;

cout << value << endl;
```

    is easier to read than this code:

```
if(num<5)
   value=num+(value-1)/2;
cout<<value<<endl;
```

  - But be careful not to insert spaces where they do not belong. You can introduce syntax errors if you do. For example, there cannot be a space between the two **<** characters that form the stream output operator **<<**.

- Blank lines should be used to separate the sections of code. For example, use a blank line after constant declarations, variable declarations, initialization sections, input sections, computation sections, and output sections. Remember, although the compiler ignores blank lines, they make code more readable for humans.

- Code logic blocks (i.e. code between { } braces) MUST be **indented** to enhance readability for humans.

- To prevent having to scroll left and right while reading program code, neither code, nor comments, should extend beyond 80 columns. Note that there is a faint grey line in the Dev-C++ IDE that shows you where this is (see instructions for Creating Homework Programs for instructions on turning this feature on in Code::Blocks).

- Use of **global variables** is ***prohibited***.
  - **Global** variables are variables that have been declared *outside/above* the function in which they are used.
  - **Local** variables are variables that have been declared *inside* the function.
  - ALL of your **variables** must be **local** variables.

©2013. Regis University

- Use of global **constants**, however, is ***encouraged*** where appropriate. Remember, a constant is identified by the keyword **const**.

- **Comments:** You should not comment **every** line of code. Instead, code should be grouped into meaningful sections and a comment should precede the code describing what the section of code does.

    - For programs that include user-defined functions (introduced later in the course), you must include a comment above each function that includes:
        - A description of what the function does
        - A description of any input parameters
        - A description of any output parameters or return values

    - In general, developers don't document within the code what is obvious. However, it may be helpful for beginning programmers to over-comment initially. Hence the following example contains more comments than necessary.

    - Comments should describe **what is happening** and **how it is being done** when it is not obvious from the code itself. *Comments that simply parrot the code are of no value*.

    - Comments should follow the indentive structure of the program and the code being commented.

Your facilitator will let you know if you're over-commenting, and will probably deduct points if you're under-commenting.

An example program, with comments, is included on the next page.

©2013. Regis University
Last modification: 1-1-2013

```
// -------------------------------------------------------
// Author: John Jones
// Class:  CS361
// Assignment: Week 1, Program 1
// Program Description: Finds the area of a circle
//   Input: Reads the radius from the user
//   Processing:  Calculates the area of circle
//   Output: Displays the radius and area
// -------------------------------------------------------

#include <iostream>           // for I/O
#include <iomanip>            // for formatting output
#include <cmath>              // for using pow function

using namespace std;

// global constant
const float PI = 3.14159;     // Approximation of Pi

// -------------------------------------------------------------
// Description: calcCircleArea calculates the area of a circle
//   Input parameter: radius - radius of the circle in inches
//   Returns: area - area of the circle in square inches
// -------------------------------------------------------------
float calcCircleArea (float radius)
{
    float area = PI * pow(radius, 2);
    return area;
}

// -------------------------------------------------------------
// Description: main reads the radius of a circle and calls
// a function to calculate the area. Displays radius and area.
//   Returns: 0 if successful
// -------------------------------------------------------------
int main()
{
    // local variables
    float radius;         // Radius input by the user (in inches)
    float area;           // Calculated area of circle

    //Read input from user
    cout << "Please input the radius of a circle (in inches): ";
    cin >> radius;
    cout << endl;

    area = calcCircleArea(radius);

    //Output the results
    cout << fixed << showpoint << setprecision(2);
    cout << "Circle radius: " << setw(10) << radius << endl;
    cout << "Circle area:   " << setw(10) << area << endl;

    cout << endl << endl;
    system("PAUSE");
    return 0;
}
```

©2013. Regis University